Lab 1: Linux, Shell, and C++ Foundations

Xuanye Xu (24020036071 / H00490299)

25 September 2026

This report documents all five assessed parts of Lab 1. An official Ubuntu 20.04.6 LTS x86_64 image was installed as a WSL 2 distribution on a Windows host and configured for command-line C++ development. Direct `g++` and CMake builds succeeded. Shell exercises produced reproducible text counts and a file containing five appended `fortune` results. Ten C++ warm-up answers were checked with a compiled program. Finally, the supplied `RandomVector` interface was implemented and tested, including histogram conservation and boundary cases. Each assessed part below identifies its method, observed result, and saved evidence.

No.	Assessed part	Points	Primary evidence
1	Ubuntu installation and setup	40	Distribution, architecture, account, and tool versions (Fig. 1); real terminal-window screenshot (Fig. 2).
2	Compile example code	5	Successful `g++` and CMake builds and executable output (Fig. 3).
3	Shell operations	5	Dante counts and five appended fortunes (Fig. 4).
4	C++ questions	25	Ten reasoned answers and compiled checks (Fig. 5).
5	C++ practice	25	Implemented class, histogram, and boundary checks (Fig. 6).

1. Ubuntu installation and setup (40 points)

The course slide requests Ubuntu 20.04 and recommends a virtual machine for classroom use. For this experiment I used an x86_64 Windows host running **WSL 2**, with a separate `Ubuntu-20.04` distribution. The existing Ubuntu 26.04 distribution was left in place. It is not a bare-metal installation and has no separate Ubuntu desktop session; Linux GUI programs are shown directly on the Windows desktop through WSLg (Figure 2). Lab 2 later ran turtlesim and RViz in the same way; the Unity simulators of Labs 3 and 4 have not been tested yet.

The official Ubuntu 20.04.6 WSL image was downloaded from the Ubuntu release site. Its SHA-256 was checked against the published `SHA256SUMS`; the full matching digest is recorded in `results/environment.txt`. The Windows `wsl.exe --list --verbose` output identified `Ubuntu-20.04` as version 2. Within that distribution, `/etc/os-release` identified Ubuntu 20.04.6 LTS and `uname -m` reported `x86_64`. The normal user `qz` has `sudo` membership and a working password, entered by the owner during first-run setup.

After updating and upgrading the package lists, I installed `build-essential`, CMake, and Git. The measured versions were `g++ 9.4.0`, `cmake 3.16.3`, `git 2.25.1`, and `GNU Make 4.2.1`. A C++ compiler smoke test built without warnings and printed `Ubuntu 20.04 toolchain OK`. Figure 1 shows a fresh verification of the OS, architecture, user, and packages. Figure 2 is

a real screenshot of an Ubuntu terminal window (xterm) opened through WSLg, in which the same checks were typed; its transcript is saved beside the image.



Figure 1: Ubuntu environment verification. This figure is typeset from live SSH / WSL command output and ıscript; it is eenshot.

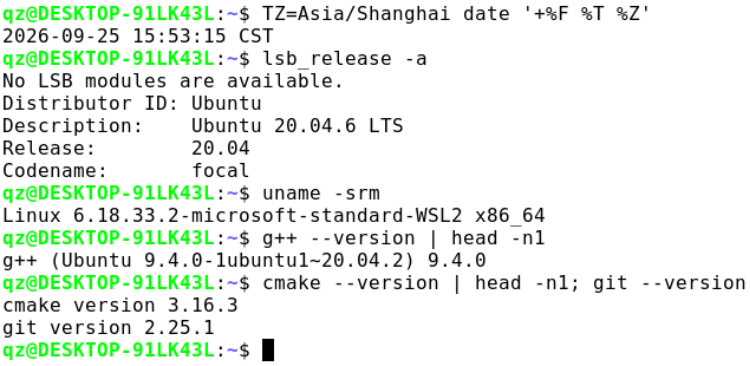


Figure 2: Real screenshot of an Ubuntu 20.04 terminal window (xterm) displayed on the Windows desktop through WSLg, 25 September 2026. The commands were typed into this interactive shell and the window was captured from the X server; transcript in `results/figures/01b_ubuntu_terminal_window.txt`.

2. *Compiling example code (5 points)*

I compiled a single-file example with C++11, warnings, and pedantic checking:

```
g++ -std=c++11 -Wall -Wextra -pedantic hello.cpp -o hello
./hello
```

The executable printed “Hello from Lab 1 on Ubuntu 20.04!”. I then configured and built a multi-file CMake example. Its `message.cpp` was compiled into a static library, and `main.cpp` linked against that library. The final executable printed `CMake library link OK`. Both build routes exited with status 0 and produced no compiler diagnostics. This contrast made the distinction between compiling

source files and linking an executable concrete.



Figure 3: Direct C++ compilation and CMake build, followed by the two actual program outputs. Rendered from live SSH/WSL output.

3. *Shell operations (5 points)*

For the text-counting exercise, the `dante.txt` file was obtained from the repository named in the MIT exercise. The original direct download did not complete in this network environment, so I used a CDN copy of the same repository file and compared it byte-for-byte with the GitHub Contents API response before counting. The saved input contains 557,042 bytes; its SHA-256 is recorded in `results/shell.txt`.

Ubuntu command	Result	Interpretation
`wc -l dante.txt`	19,567	Newline-terminated lines.
`wc -w dante.txt`	97,676	Words under the Ubuntu locale used for the exercise.
`grep -cve '^[[:space:]]*$' dante.txt`	14,338	Lines containing non-whitespace text.

For output redirection, a script created `fortunes.txt` once and executed `/usr/games/fortune >> fortunes.txt` five times. The cumulative byte totals after each append were 53, 175, 365, 388, and 441. The final file is 441 bytes. Its nine text lines do not imply nine invocations: a single fortune may span multiple lines. I also exercised `mkdir`, `cd`, `pwd`, copying, moving, `chmod`, `find`, pipes, and `wc`. Figure 4 records the counts and the append script. The original text and script remain available for inspection.

The word count differed under the default macOS locale (97,765) even though the bytes were identical; `LC_ALL=C` on macOS gave 97,676. Therefore the submitted result is explicitly the value measured in Ubuntu, with the command and environment stated.



Figure 4: Dante line/word counts, nonblank-line count, five-append script, and final fortune-file size and hash. Rendered from live SSH/WSL output.

4. C++ warm-up questions (25 points)

I answered all ten questions in the order of the MIT warm-up exercise. The short conclusions below are paired with the reasoning needed to explain them at an experiment check. The complete answer file is `lab1/cpp-warmup.txt`; `cpp_warmup_check.cpp` executes the corresponding snippets.

O1 — Pre- and post-increment. After `j = ++i;` and then `j = i++;`, $i = 2$ and $j = 1$. The first assignment uses the incremented value; the second assigns the old value before incrementing.

O2 — Conditional operator. The expression `42 < 42` is false, so the program prints `b`.

R1 — Reference alias. The output is `10 10`. The reference and original integer name the same object; writing through the reference changes the original.

R2 — Pointer dereference. The expression `*j**j` parses as `(*j) * (*j)`, not exponentiation. It stores $42 \times 42 = 1764$ through the pointer and prints `1764`.

R3 — Pointer arithmetic. `i + 2` addresses the third array element. The assignment uses the second and fourth elements, $24 - 24 = 0$, so the program prints `0`.

R4 — Reference parameter. The function's `int&` parameter aliases the caller's `j`. Resetting it to zero makes the program print `0`.

N1 — Integer types. `short`, `int`, `long`, and `long long` have nondecreasing required ranges, but byte widths depend on the ABI. On this Ubuntu x86_64 installation, `sizeof` measured 2, 4, 8, and 8 bytes respectively. These are measurements, not portable constants.

N2 — Floating-point and conversion. `double` provides at least the precision/range of `float`; here they occupy 8 and 4 bytes respectively. Assigning the `double` literal `3.14` to an `int` discards the

fractional part, so $i = 3$.

N3 — Unsigned conversion. With 8-bit `unsigned char`, converting -1 is modulo 256, so $c = 255$. The assumption about character width is essential.

N4 — Boolean condition. Nonzero `42` converts to true. The first branch sets $i = 0$; the `else` branch is skipped.

The compiled run in Figure 5 agreed with every predicted value. The source file and plain-text output are kept alongside the report.

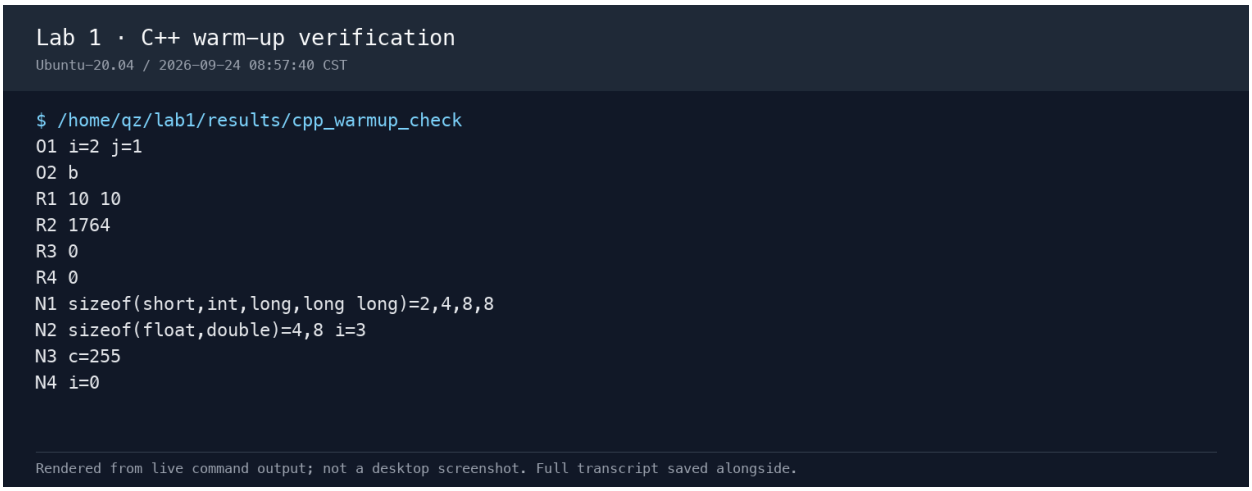


Figure 5: Compiled verification of all ten C++ warm-up answers, including platform-specific type sizes. Rendered from live SSH/WSL output.

5. C++ practice: RandomVector (25 points)

I kept the original MIT class interface and demonstration entry point and filled the six methods in `random_vector.cpp`. The constructor generates `size` doubles by scaling `std::rand()` with a floating-point divisor, avoiding integer truncation:

$$x_i = \frac{\text{rand}()}{\text{RAND_MAX}}\,\texttt{max_val}, \qquad 0 \le x_i \le \texttt{max_val}.$$

The demonstration seeds the generator with 314159. The `print` method emits every value; `mean`, `min`, and `max` use simple loops and no functions from `<algorithm>`.

For B histogram bins between observed minimum m and maximum M, the index is

$$b(x) = \min\left(\left\lfloor B\frac{x-m}{M-m}\right\rfloor, B-1\right), \qquad M > m.$$

Thus regular bins are left-closed and right-open, while the last bin includes the maximum. For $M = m$, all values go into the first bin to avoid division by zero. The histogram is printed as vertical columns. Invalid size, nonfinite or negative upper bound, and nonpositive bin count raise `std::invalid_argument`.

The Ubuntu default run produced 20 values with mean 0.55804, minimum 0.0278694, and maximum 0.982655. Its five column counts

were $5, 1, 4, 3, 7$, summing exactly to 20. Figure 6 shows both the actual histogram and the verification summary. The test program also checked the printed value count and range, independent bin classification from the seeded inputs, a singleton, zero and scaled upper bounds, one bin, more bins than values, and invalid arguments. All passed on Ubuntu 20.04.

The Ubuntu sequence differs from the MIT sample and a local macOS run. A fixed seed reproduces the sequence within one `std::rand()` implementation, but the C library does not promise the same sequence across platforms. The requirements are therefore checked by range, statistics, and bin conservation rather than by copying the sample numbers.

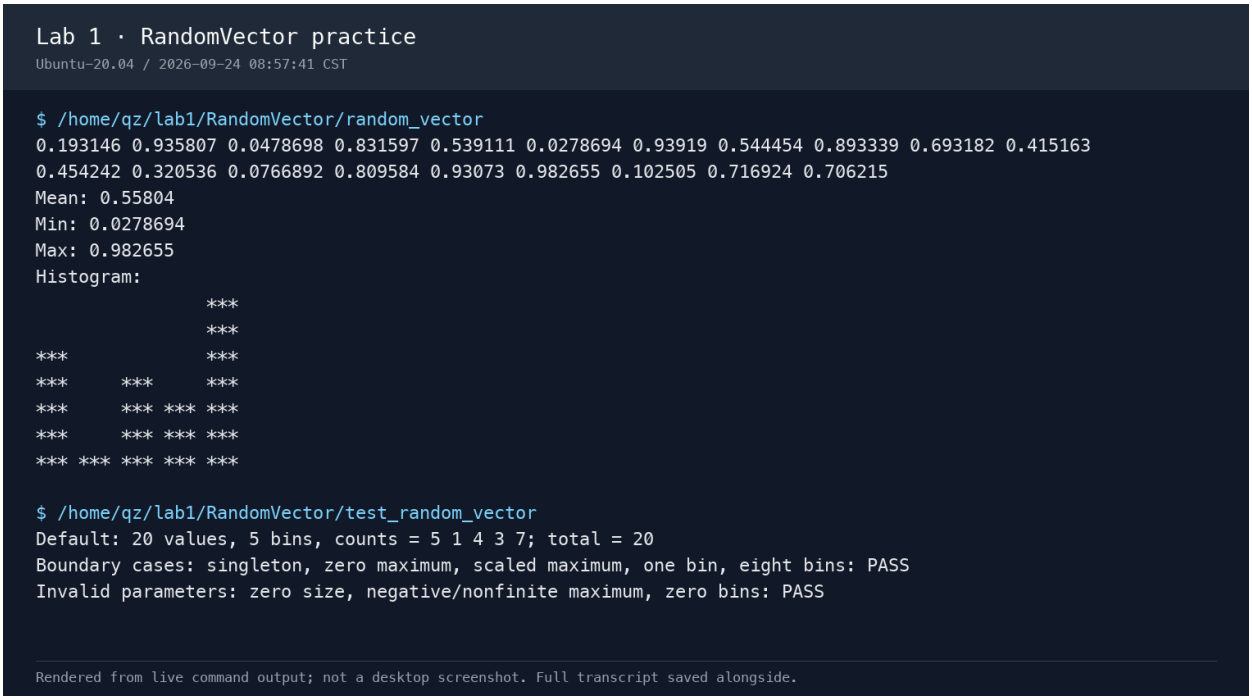


Figure 6: `RandomVector` default output and boundary verification on Ubuntu 20.04. Rendered from live SSH/WSL output.

Discussion and reproducibility

The two cross-platform discrepancies were instructive. The same Dante bytes gave different default word counts on macOS and Ubuntu because the counting locale differed. The same C++ random seed gave different values because `std::rand()` implementations differed. Both cases show why an experimental result needs its command, environment, and input, not just its final number. The histogram required an explicit upper-bound rule: without clamping the maximum to the last bin, the natural formula can index past the array. These were verified with the original inputs and targeted boundary checks rather than inferred from the example output.

WSL 2 supported all Lab 1 command-line tasks, and WSLg displays Linux GUI windows (Figure 2). The Unity simulators of later

labs still need their own validation.

To reproduce the C++ practice from `lab1/RandomVector` on Ubuntu:

```
g++ -std=c++11 -Wall -Wextra -pedantic \
  main.cpp random_vector.cpp -o random_vector
./random_vector
g++ -std=c++11 -Wall -Wextra -pedantic \
  test_random_vector.cpp random_vector.cpp \
  -o test_random_vector
./test_random_vector
```

The answer file, source code, and command transcripts accompany this PDF in the Lab 1 directory. Each figure renders saved live SSH/WSL output with an adjacent text transcript; it is not a native window screenshot.

References

1. Course materials, *Lab 1* slides and *Course Syllabus*, supplied in this project.
2. MIT VNAV, *Lab 1 Exercises*, `https://vnav.mit.edu/labs_2023/lab1/exercises.html`.
3. MIT-SPARK, *VNAV-labs Lab 1 source template*, `https://github.com/MIT-SPARK/VNAV-labs/tree/master/lab1`.
4. Ubuntu, *Ubuntu 20.04.6 WSL image and SHA256SUMS*, `https://releases.ubuntu.com/focal/`.